Lecture 05.06: Attributes — Class vs Instance Attributes

📌 Executive Summary

Class-level attributes vs instance-level attributes, namespace lookup hierarchy, attribute shadowing trap, and memory economics.

🔬 Theoretical Foundations & Principles

1. Namespace Hierarchy

- **Class Attributes:** Defined directly in the class body; stored in `Class.__dict__`; shared across all instances.
- **Instance Attributes:** Defined on `self`; stored in `instance.__dict__`; unique to each individual object.

2. The Shadowing Trap

Reading `obj.attr` first checks `obj.__dict__`. If absent, it checks `Class.__dict__`. Writing `obj.attr = value` always creates a new entry in `obj.__dict__`, "shadowing" the class attribute without mutating it!

$$\text{Lookup}(o, x) = \begin{cases} o.__\text{dict}__[x] & \text{if } x \in o.__\text{dict}__ \\ C.__\text{dict}__[x] & \text{else if } x \in C.__\text{dict}__ \end{cases}$$

💻 Production Implementation & Code

```
class Employee:
    company_name = "AI Corp"  # Class attribute (shared)

    def __init__(self, name: str):
        self.name = name      # Instance attribute (unique)

e1 = Employee("Alice")
```

```
e2 = Employee("Bob")

print("Company E1:", e1.company_name)
print("Company E2:", e2.company_name)

# Mutating via class updates all instances
Employee.company_name = "Global AI Inc"
print("Updated E1:", e1.company_name)
print("Updated E2:", e2.company_name)
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.